dtorrent_task_v2 0.4.4

Published 1 hour ago

SDK: DART FLUTTER | PLATFORM: ANDROID IOS LINUX MACOS WINDOWS

0

Readme | Changelog | Installing | Versions | Scores

0 LIKES | 140 POINTS | 87 DOWNLOADS

Publisher
unverified uploader

Weekly Downloads

2024.12.11 - 2025.11.09

Metadata

BitTorrent download client package written by pure Dart language.

Repository (GitHub)
View/report issues

Documentation

API reference

License

BSD-3-Clause (license)

Dependencies

args, b_encode_decode, bittorrent_dht, collection, crypto, dart_ipfs, dtorrent_common, dtorrent_parser, dtorrent_task, events_emitter2, logging, mime, utp_protocol

More

Packages that depend on dtorrent_task_v2

About

Dart library for implementing BitTorrent client.

The Dart Torrent client consists of several parts:

- Bencode
- Tracker
- DHT
- Torrent model
- Common library
- UTP

This package implements the regular BitTorrent Protocol and manages the above packages to work together for downloading.

BEP Support

- BEP 0003 The BitTorrent Protocol Specification
- BEP 0005 DHT Protocol
- BEP 0006 Fast Extension
- BEP 0009 Extension for Peers to Send Metadata Files
- BEP 0010 Extension Protocol
- BEP 0011 Peer Exchange (PEX)
- BEP 0012 Multitracker Metadata Extension
- BEP 0014 Local Service Discovery
- BEP 0015 UDP Tracker Protocol
- BEP 0019 HTTP/FTP Seeding (GetRight-style)
- BEP 0027 Private Torrents
- BEP 0029 uTorrent transport protocol
- BEP 0053 Magnet URI extension - Select specific file indices
- BEP 0055 Holepunch extension

How to use

This package requires dependency `dtorrent_parser`:

```
dependencies:
  dtorrent_parser: ^1.0.0
  dtorrent_task_v2: ^0.4.4
```

Download from: DTORRENT_TASK_V2

Import the library:

```
import 'package:dtorrent_task_v2/dtorrent_task_v2.dart';
```

First, create a `Torrent` model from a .torrent file:

```
var model = await Torrent.parse('some.torrent');
```

Second, create a `Torrent Task` and start it:

```
var task = TorrentTask.newTask(model, 'savepath');
await task.start();
```

You can add event listeners to monitor `TorrentTask` execution:

```
EventsListener<TaskEvent> listener = task.createListener();
listener
  ..on<TaskCompleted>((event) {
    print('Download completed!');
  })
  ..on<TaskFileCompleted>((event) {
    print('File completed: ${event.file.originalFileName}');
  })
  ..on<TaskStopped>((event) {
    print('Task stopped');
  });
```

And there are methods to control the `TorrentTask`:

```
// Stop task:
await task.stop();
// Pause task:
task.pause();
// Resume task:
task.resume();
```

Using Magnet Links

The library supports downloading from magnet links with automatic metadata download:

```
import 'package:dtorrent_task_v2/dtorrent_task_v2.dart';

// Parse magnet link
final magnet = MagnetParser.parse('magnet:?xt=urn:btih:...');
if (magnet == null) {
  print('Invalid magnet URI');
  return;
}

// Download metadata first
final metadata = MetadataDownloader.fromMagnet('magnet:?xt=urn:btih:...');
final metadataListener = metadata.createListener();

metadataListener
  ..on<MetaDataDownloadProgress>((event) {
    print('Metadata progress: ${(event.progress * 100).toInt()}%');
  })
  ..on<MetaDataDownloadComplete>((event) {
    print('Metadata downloaded!');
    // Parse torrent from metadata
    final msg = decode(event.data);
    final torrentMap = <String, dynamic>{'info': msg};
    final torrentModel = parseTorrentFileContent(torrentMap);

    if (torrentModel != null) {
      // Start download with web seeds and selected files from magnet link
      final task = TorrentTask.newTask(
        torrentModel,
        'savepath',
        false, // stream
        magnet.webSeeds.isNotEmpty ? magnet.webSeeds : null,
        magnet.acceptableSources.isNotEmpty ? magnet.acceptableSources : null,
      );

      // Apply selected files from magnet link (BEP 0053)
      if (magnet.selectedFileIndices != null &&
          magnet.selectedFileIndices!.isNotEmpty) {
        task.applySelectedFiles(magnet.selectedFileIndices);
      }

      await task.start();

      // Transfer peers from metadata downloader to avoid reconnection delays
      final metadataPeers = metadata.activePeers;
      for (var peer in metadataPeers) {
        task.addPeer(peer.address, PeerSource.manual, type: peer.type);
      }

      // Add trackers from magnet link
      if (magnet.trackers.isNotEmpty) {
        final infoHashBuffer = Uint8List.fromList(
          List.generate(magnet.infoHashString.length ~/ 2, (i) {
            final s = magnet.infoHashString.substring(i * 2, i * 2 + 2);
            return int.parse(s, radix: 16);
          }),
        );
        for (var trackerUri in magnet.trackers) {
          task.startAnnounceUrl(trackerUri, infoHashBuffer);
        }
      }
    }
  });

metadata.startDownload();
```

Advanced Features

Web Seeding (BEP 0019)

The library supports HTTP/FTP seeding from web seed URLs specified in magnet links:

```
// Web seeds are automatically used when no peers are available for a place
// They are specified in magnet links with the "ws" parameter:
// magnet:?xt=urn:btih:...&ws=http://example.com/file.torrent

final task = TorrentTask.newTask(
  torrentModel,
  'savepath',
  false,
  [Uri.parse('http://example.com/file.torrent')], // webSeeds
  null, // acceptableSources
);
```

Selected Files (BEP 0053)

You can download only specific files from a torrent:

```
// Select files by index (0-based)
task.applySelectedFiles([0, 2, 5]); // Only download files at indices 0, 2, and 5

// This is especially useful with magnet links:
// magnet:?xt=urn:btih:...&so=0,2,5
```

Monitoring Download Progress

You can monitor detailed download progress:

```
final listener = task.createListener();
listener.on<StateFileUpdated>((event) {
  final downloaded = task.downloaded ?? 0;
  final progress = task.progress * 100;
  final connectedPeers = task.connectedPeersNumber;
  final seederCount = task.seederNumber;
  final downloadSpeed = task.currentDownloadSpeed;
  final avgSpeed = task.averageDownloadSpeed;

  print('Progress: ${progress.toStringAsFixed(2)}%');
  print('Downloaded: ${(downloaded / 1024 / 1024).toStringAsFixed(2)} MB');
  print('Peers: $connectedPeers ($seederCount seeders)');
  print('Speed: ${(downloadSpeed * 1000 / 1024).toStringAsFixed(2)} KB/s');
  print('Avg: ${(avgSpeed * 1000 / 1024).toStringAsFixed(2)} KB/s');
});
```

Adding Peers Manually

You can add peer addresses manually:

```
import 'package:dtorrent_common/dtorrent_common.dart';

// Add a peer by address
final peerAddress = CompactAddress(
  InternetAddress('192.168.1.100'),
  6881,
);
task.addPeer(peerAddress, PeerSource.manual, type: PeerType.TCP);

// Add a peer with existing socket
task.addPeer(peerAddress, PeerSource.incoming,
    type: PeerType.TCP, socket: socket);
```

DHT Support

The library includes built-in DHT support for peer discovery:

```
// DHT is automatically enabled in TorrentTask
// You can add bootstrap nodes:
for (var node in torrentModel.nodes) {
  task.addDHTNode(node);
}

// Or manually request peers from DHT:
task.requestPeersFromDHT();
```

Monitoring and Error Tracking

The library includes comprehensive error tracking for uTP protocol stability:

```
import 'package:dtorrent_task_v2/src/peer/protocol/peer.dart';

// Check RangeError metrics
if (Peer.rangeErrorCount > 0) {
  print('Total RangeErrors: ${Peer.rangeErrorCount}');
  print('uTP RangeErrors: ${Peer.utpRangeErrorCount}');
  print('Errors by reason: ${Peer.rangeErrorByReason}');
}

// Reset metrics for new monitoring period
Peer.resetRangeErrorMetrics();
```

These metrics help monitor uTP protocol stability and debug RangeError crashes, particularly those related to selective ACK processing and buffer handling.

Features

Stability Improvements

- uTP RangeError Protection: Comprehensive protection against RangeError crashes in uTP protocol with:
  - Buffer bounds validation before all operations
  - Message length validation (negative, oversized, and overflow protection)
  - Integer overflow protection in calculations
  - Detailed error tracking and metrics
  - Extensive test coverage (stress tests, reordering, extreme values, long sessions)
- Critical Bug Fixes: Fixed race condition in bitfield processing that prevented downloads from starting (see issue #4)

Protocol Support

- Full BitTorrent protocol implementation
- uTP (uTorrent transport protocol) support with enhanced stability
- TCP fallback support
- Multiple extension protocols (PEX, LSD, Holepunch, Metadata Exchange)
- Magnet link support via `MagnetParser` with automatic metadata download
- Torrent creation via `TorrentCreator`
- Web seeding (HTTP/FTP) support (BEP 0019)
- Selected file download (BEP 0053)
- Private torrent support (BEP 0027) with automatic DHT/PEX disabling

Performance

- Efficient piece management and selection
- Memory-optimized file handling
- Streaming support for media files with isolate-based processing
- Optimized congestion control for uTP connections
- Debounced progress events for reduced UI update frequency
- Automatic peer transfer from metadata download to actual download
- Metadata caching to avoid re-downloading
- Parallel metadata download from multiple peers

Magnet Link Features

- Automatic metadata download from magnet links
- Support for Base32 and hex infohash formats (RFC 4648)
- Tracker tier support (BEP 0012) with tier-by-tier announcement
- Web seed URL parsing and integration (BEP 0019)
- Acceptable source URL support
- Selected file indices parsing (BEP 0053)
- Automatic peer and tracker transfer from metadata phase to download phase